

```
<!DOCTYPE html> <html>
<head> <meta charset
="UTF-8"> <title>Sample
Purchase Order</title> <l
ink rel="stylesheet" href
="https://cdnjs.cloudflare...
```

PASTED

in zoho creator i want to upload multiple image for a viedl can cotain 10 image
https://<base_url>/creator/v2.1/data/<account_owner_name>/<app_link_na
me>/report/<report_link_name>/<record_ID>/<field_link_name>/upload

Architected multi-file upload solution with rate-limit constraints >

Good news: this is doable with `ZOHODESK.request()`, but there are two important constraints from Zoho's docs that change the flow compared to your current JSON-only submit.

1. The upload endpoint is one-file-per-call

Per Zoho's Upload File API v2.1: when a field has "multiple files" enabled, *each file must be uploaded in a separate API call* — there's no batch/array upload to a single field in one request. So for 10 images you'll make 10 separate calls to:

```
https://www.zohoapis.com/creator/v2.1/data/towngroup/fashion-
visit/report/<report_link_name>/<record_ID>/<field_link_name>/upload
```

Each call needs a `record_ID`, so you must create the `Sample_Purchase` record **first** (your existing form POST), grab the new record's ID from the response, then loop the image uploads against that ID.

2. ZOHODESK.request() file uploads are capped at 3/minute

The extension SDK docs explicitly state: *"You can upload a maximum of 3 files per minute"* via the Request Method. For 10 images that means you need throttling — roughly 3-4 minutes total if done politely. You'll want a progress indicator so the user isn't staring at a frozen spinner.

Here's how I'd restructure your submit logic:

javascript

```
async function submitPurchaseOrder() {
  // ... your existing validation stays the same ...

  document.getElementById('loading').style.display = 'block';

  try {
    const dataToSend = {
      "Purchase_Num": parseInt(purchaseNum),
      "Item_ID": selectedItem.ID,
      "Item_Name": selectedItem.Item,
      "Date_field": purchaseDate,
      "Purchase_Qty": parseInt(purchaseQty),
      "Year_field": yearField,
      "Brand": brand,
      "Season": season,
      "Country": country,
      "Payment_Type": paymentType,
      "Last_4_Digit_from_Visa": last4Digits,
      "Cost": parseFloat(cost),
      "Comment": comment,
      "Ticket_Id": currentTicketID
    };

    const requestObj = {
```

```
url: "https://www.zohoapis.com/creator/v2.1/data/towngroup/fashion-visit",
headers: { 'Content-Type': 'application/json' },
type: 'POST',
postBody: JSON.stringify({ data: dataToSend }),
connectionLinkName: "creator"
};

const response = await ZOHODESK.request(requestObj);
const parsed = typeof response === 'string' ? JSON.parse(response) : response;
console.log("Create response:", parsed); // confirm the shape once, then remove

const newRecordID = parsed.data?.ID;
if (!newRecordID) throw new Error("Couldn't read new record ID from response");

if (receiptImages.length > 0) {
    await uploadReceiptImages(newRecordID);
}

showToast("Purchase order submitted successfully!", 'success');
resetForm();

} catch (error) {
    console.error("Error:", error);
    showToast("Error submitting purchase order: " + error.message, 'error');
} finally {
    document.getElementById('loading').style.display = 'none';
}
}

async function uploadReceiptImages(recordID) {
    const reportLinkName = "Sample_Purchase_Report"; // replace with the actual report link name
    const fieldLinkName = "Receipt_Images"; // replace with your image field link name

    for (let i = 0; i < receiptImages.length; i++) {
```

```

const file = receiptImages[i];
setLoadingText(`Uploading image ${i + 1} of ${receiptImages.length}...`);

const uploadObj = {
  url: `https://www.zohoapis.com/creator/v2.1/data/towngroup/fashion-visit`,
  type: "POST",
  data: {},
  postBody: {},
  headers: { "Content-Type": "multipart/form-data" },
  fileObj: [{ key: "file", file: file }],
  connectionLinkName: "creator"
};

try {
  await ZOHODESK.request(uploadObj);
} catch (err) {
  console.error(`Image ${i + 1} failed:`, err);
  // optionally surface a non-blocking toast and continue
}

// throttle: 3 file uploads/min limit on the Request method
if (i < receiptImages.length - 1 && (i + 1) % 3 === 0) {
  setLoadingText("Pausing for upload rate limit (60s)...");
  await new Promise(res => setTimeout(res, 61000));
}
}

function setLoadingText(msg) {
  const el = document.querySelector('#loading .loading-content p');
  if (el) el.innerHTML = `<i class="fas fa-sync-alt fa-spin"></i> ${msg}`;
}

```

A few things to set up on the Creator side that aren't visible from the widget code:

The target field in `Sample_Purchase` needs to be an Image or File Upload field with "Allow multiple files" turned on and "Maximum number of files" set to 10 — that's what actually enforces the 10-image cap, the API itself just returns an error once the field is full.

The "creator" connection needs the `ZohoCreator.report.CREATE` scope (in addition to whatever read scope you're already using for `Items_Report`), or the upload calls will fail with a permission error.

Double-check `reportLinkName` — link names for reports often differ from form names (e.g. an "All Sample Purchases" report might have a link name like `All_Sample_Purchases` rather than `Sample_Purchase_Report`). You can find it in Creator's builder under that report's properties panel.

One UX caveat worth flagging to whoever's using this form: with the 3-files/minute cap, 10 images realistically takes 3–4 minutes after they hit submit. If that feels too slow, an alternative is to have the widget hand the record ID + files off to a Deluge custom function (via the same "creator" connection but using `invokeurl` with `files:`), which doesn't carry this particular SDK-level throttle — but that's a bigger architectural change involving deploying a custom function in the Creator app. Happy to sketch that out if the 3-minute wait turns out to be a dealbreaker.

